LaTeX Showcase

1 Overview

This is a quick showcase of the best (read: flashiest) LaTeX features. The source code for this document can be found here. Its two main pulls (generally) are dynamic control and vectorisation. A LaTeX project is broadly composed of preamble and body content (which may be split across multiple files for easier organisation on large documents). The preamble of a document allows you to define header, footer, and section styles, colour palettes, and any other holistic document formatting easily, so revision-specific changes are easy to implement, signpost, and revert. Everything can be programmatically defined with commands and counters in the preamble. For example, all section, table, and figure labels are customisable and referenced dynamically with `\ref{item:<item_name>}`[1] so we never need to go through an entire document and reindex if, say, a section is split into two or a new figure is added. Tables of contents are also dynamic because of this. Labels can be defined for anything, so dynamically referencing anything specific (like requirements) is possible, too. Citations are dynamically imported from a bibliography file, so something like changing referencing style is equally trivial. All of this makes changing larger projects easy, because the tedious work is automatically reevaluated.

Tex typesetting is also known for its maths:

$$\int_a^b f(x)\mathrm{d}x = F(b) - F(a)$$

Which is well deserved. We can have inline, too: for $z \in \mathbb{C}, z = x + iy \; \forall \; x, y \in \mathbb{R}$:

$$\begin{aligned}\sinh z &= \sum_{n=0}^{\infty} \frac{z^{2n+1}}{(2n+1)!} \\ &= z + \frac{z^3}{3!} + \frac{z^5}{5!} + ...\end{aligned}$$

Finally, everything is vectorised so can be scaled to any resolution. This is handy for graphics.

[1] We `\label` each one, so `\label{sec:introduction}` is referenced with `\ref{sec:introduction}`.

1.1 Theming

Since everything is centrally defined in the preamble, we can set certain styles that can be tweaked locally and applied globally. Some good examples of this are section numbering styles:

```
\renewcommand{\thesection}{\Roman{section}.}
```

Which would section numbers to be uppercase roman numerals; custom caption styles:

```
\usepackage{caption}
\DeclareCaptionFormat{custom}{
  \textsc{#1#2} \\ #3
}
\captionsetup{
  format=custom,
  font={it, stretch=1.2},
  labelsep=none
}
```

Which would set captions to be the environment type[2] in Small Caps, the number, and then a newline with the caption contents in italic; and global palettes:

```
\definecolor{red}{rgb}{1, 0.2, 0.2}
\definecolor{green}{rgb}{0.2, 0.7, 0.2}
\definecolor{blue}{rgb}{0.2, 0.2, 1}
```

Which can then be used throughout the document, and will be updated if the palette is changed.

2 Graphics

Packages like `tikzpicture` and `pgfplots` allow for dynamic, vectorised illustrations. They give a coordinate space that can be drawn on, and axes environments for plotting and analysing functions.

[2] That is, a figure, image, table, etc. This is inferred by the compiler automatically.

2.1 Tikz

Tikz is the main graphics package for creating diagrams, and gives an environment similar to Python's `turtle`. We draw on a grid, and also have access to fancy shapes. Here's a simple Tikz example with code:

```
\begin{figure}[H] % makes a box on the page
    \centering % centers the contents in the box
    \begin{tikzpicture} % start a drawing
        \draw (0,0) circle (1);
        \draw[fill=black] (0,0) circle (0.05);
        \draw (2,0) circle (1);
        \draw[fill=black] (2,0) circle (0.05);
        \draw[dashed] (0,0)--(2,0);
        \node at (1, -1.3) {$s = 2r$}; % text captions

        \draw (5,0) circle (1);
        \draw[fill=black] (5,0) circle (0.05);
        \draw (6,0) circle (1);
        \draw[fill=black] (6,0) circle (0.05);
        \draw[dashed] (5,0)--(6,0);
        \node at (5.5,-1.3) {$s = r$};

        \draw (9,0) circle (1);
        \draw[fill=black] (9,0) circle (0.05);
        \node at (9, -1.3) {$s = 0$};
    \end{tikzpicture} % close the drawing
    \caption{Intersection of equivalent circles over $s = [0, 2r]$}
    \label{fig:simple_domain} % label for referencing
\end{figure} % close the diagram
```

Which produces:

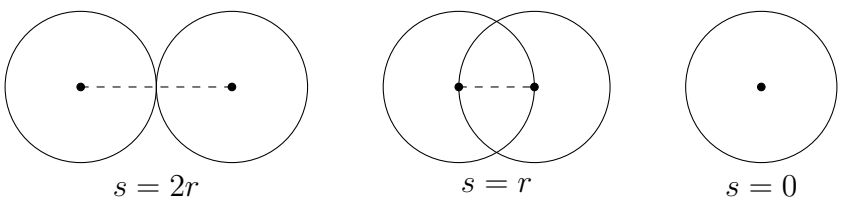


FIGURE 1

Intersection of equivalent circles over $s = [0, 2r]$

2.2 Images

Including images in Latex is straightforward, and we can use `tikz` in conjuction with images to annotate graphics easily and dynamically. We can include an image with the `\includegraphics{path/to/image.png}` command, where the image directory is specified in the preamble:

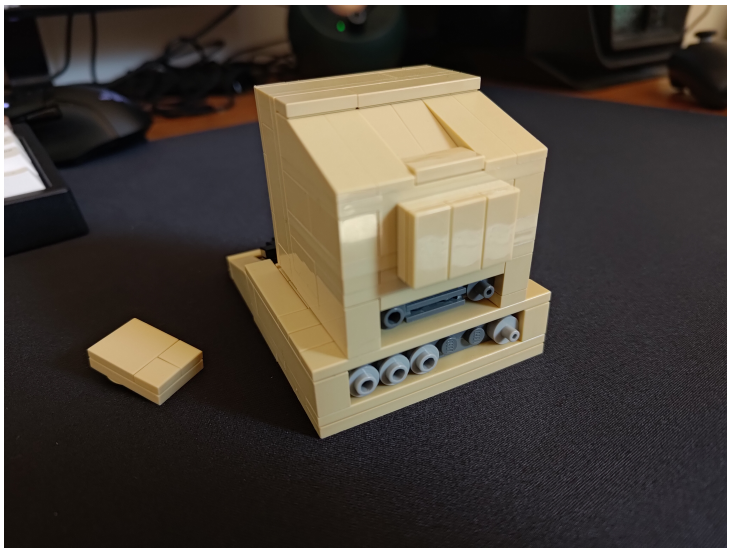


FIGURE 2
A Lego model of a BBC Micro with Archimedes CRT, from behind.

Now, if we embed this in the `tikz` environment, we can annotate it.

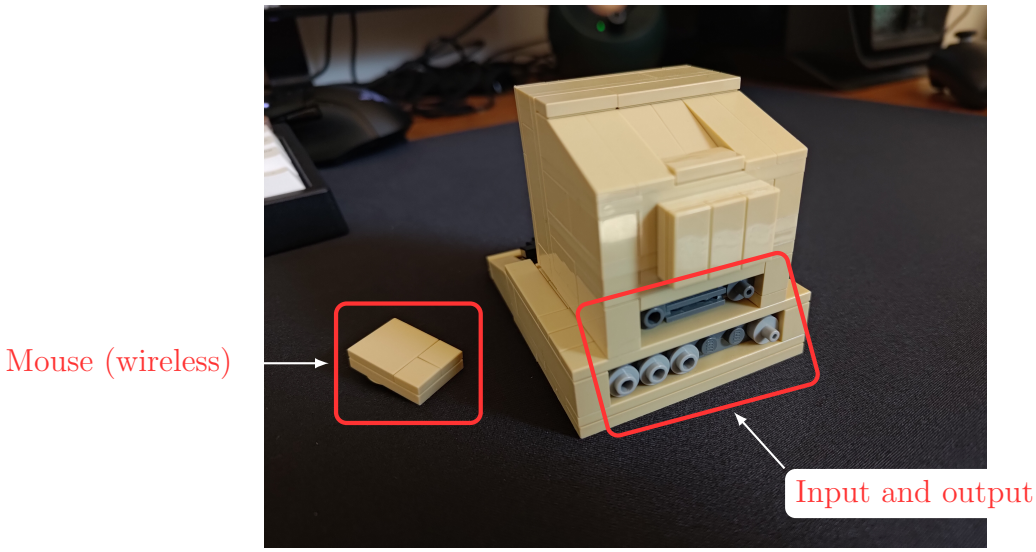


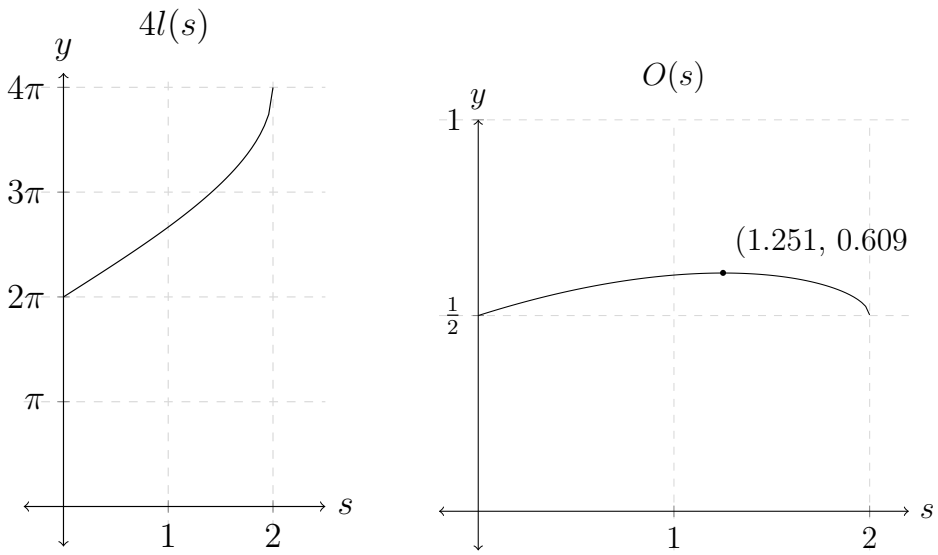
FIGURE 3
The same model, annotated.

These can now be dynamically scaled, and all of the annotations will scale with it, while

retaining control over their contents. This is superior to externally created or rasterised graphics, as all changes can be made within the `.tex` file.

2.3 Axes

The `pgfplots` packages gives the `axis` environment within `tikz`, which allows for plotting points and closed form functions explicitly.



(A)
$4l(s) : [0, 2] \to [2\pi, 4\pi]$

(B)
Result of $O'(s) = 0$

FIGURE 4
Examples for `pgfplots`

Since all of these libraries, and many others (perhaps for drawing circuits, or flow charts) plug in to Tikz, all of these features can be used in conjunction with each other, natively. A plot can be annotated, an image can be iterated over, graphs can be dynamically recoloured. This is the advantage of having one centralised system to control development.

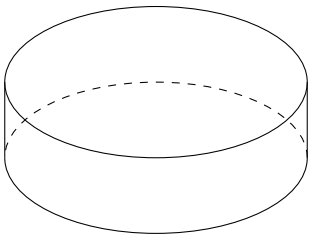
3 Commands

Latex also provides programmatic ways of working with documents. The most useful aspect of this is defining specific macros, or `commands`, in the preamble. These let you program functions that are needed for a document, and can be called wherever they

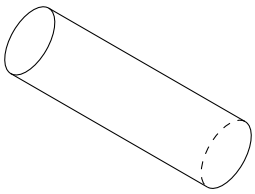
have been defined to be valid (ie. the body, footnotes, figures). As an example, we can define a command to draw, inside the Tikz environment, a cylinder with some specified dimensions and rotation:

```
\newcommand{\cylinder}[3]{ % \cylinder takes three arguments
  \def\radius{#1} % take radius
  \def\height{#2} % take height
  \def\rotation{#3} % take rotation
  \def\eccentricity{0.5} % eccentricity of circular

  % sides
  \draw[rotate=\rotation] (-\radius, -\height/2)--(-\radius, \height/2);
  \draw[rotate=\rotation] (\radius, -\height/2)--(\radius, \height/2);

  % circular faces
  \draw[rotate=\rotation] (0,\height/2) ellipse [x radius = \radius, y
  ↪ radius = \eccentricity*\radius];
  \draw[dashed, rotate=\rotation] (\radius, -\height/2) arc
  ↪ (0:180:{\radius} and {\eccentricity*\radius});
  \draw[rotate=\rotation] (-\radius, -\height/2) arc (180:360:{\radius}
  ↪ and {\eccentricity*\radius});
}
```

We can now call this in the `tikzpicture` environment with different passed values, so `\cylinder{2}{1}{0}` draws a cylinder of radius 2, height 1, and no rotation[3]:



And `\cylinder{0.5}{3}{60}` gives



[3]Tikz has a standard unit for the increments on the grid, but we can also use centimeters, inches, em, points, or pixels instead.

We also have access to flow control, iteration, and binary conditionals, so we could (if we wanted to) add an `origin` argument to the function, and iterate over some x and y values to draw cylinders with different rotations iff x and y are both prime. This can be quite powerful.

4 Gallery

Here is a collection of other Tikz graphics to show off some more complex examples.

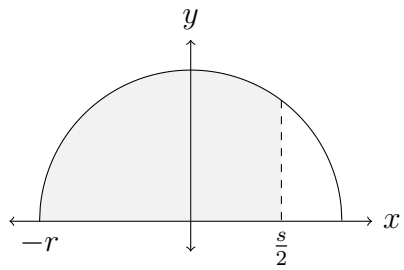


FIGURE 5
Integrated area (shaded).

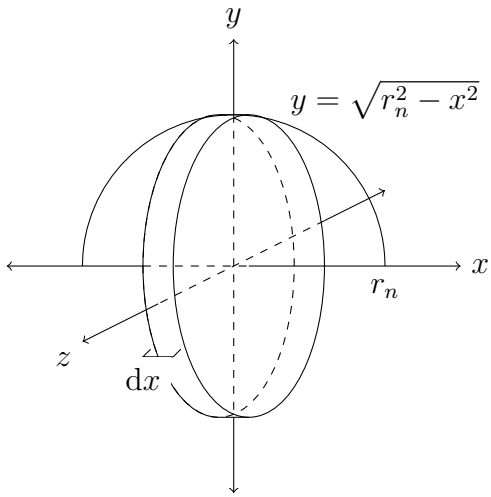


FIGURE 6
Slice of volumetric integration on $\mathrm{d}x$.

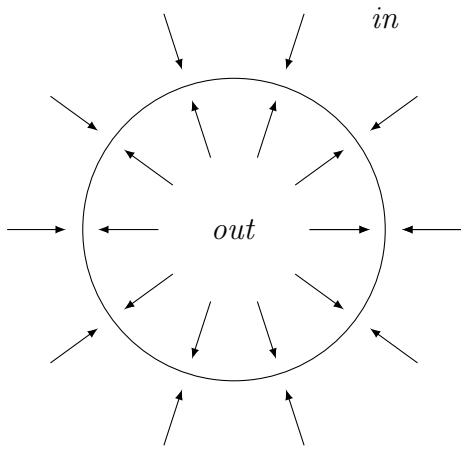


Figure 7

Iterated pressure vectors of a bubble.

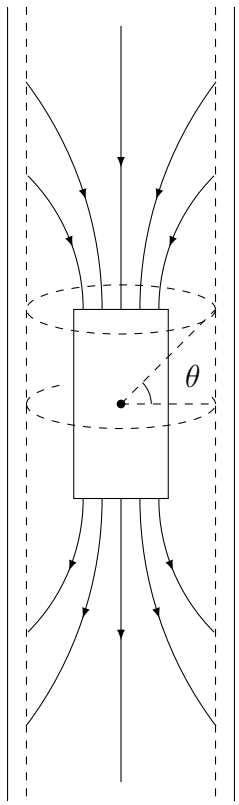


Figure 8

Cross-sectional view of the magnetic field lines in a metal pipe.

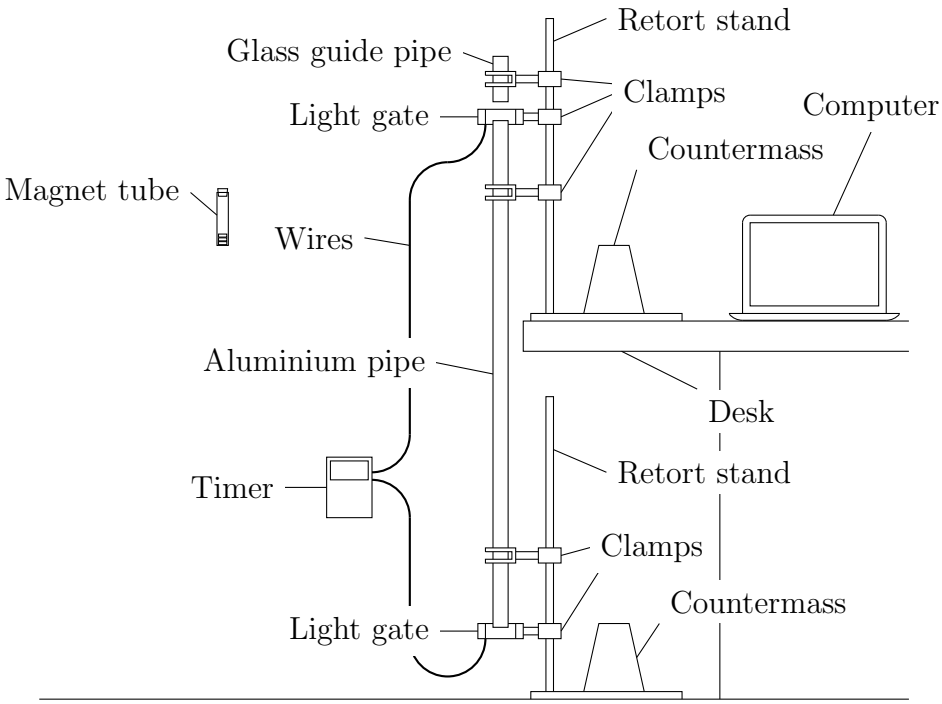


Figure 9

Arrangement of equipment.